Model examen

1. Construiți o Mașină Turing care să determine dacă un input conține un număr par de 0.

Ex: M[1001010] → TRUE
M[1010110] → FALSE

q_e → even
q_o → odd

	0	1	□
q_e	q_o, 0, →	q_e, 1, →	Y, □, -
q_o	q_e, 0, →	q_o, 1, →	N, □, -

2. Demonstrați că problema f este nedecidabilă, folosind reducerea Turing la o problemă studiată la curs. (HALT)

$$f(M_1, M_2) = \begin{cases} TRUE, & M_1 \text{ acceptă exact cuvintele respinse de } M_2 \\ & \text{(și reciproca)} \\ FALSE, & \text{altfel} \end{cases}$$

$HALT \leq_m f \Rightarrow f \notin R$
$HALT \notin R$

$\exists\, t : \Sigma^* \to \Sigma^*$, $\begin{cases} t \text{ computabilă} \\ \forall w,\ HALT(w) = TRUE \Leftrightarrow f(t(w)) = TRUE \end{cases}$

dacă scrii doar porția asta, poți lua punctaj parțial

$M_1[v]$:
run $M[w]$
Y

$M_2[v]$:
N

$\Rightarrow$" $HALT(M, w) = TRUE \Rightarrow M[w]$ halts $\Rightarrow \forall v \in \Sigma^*$;
$M_1[v] \to TRUE$ și $M_2[v] \to FALSE \Rightarrow f(M_1, M_2) = TRUE$

$\Leftarrow$" $f(M_1, M_2) = TRUE \Rightarrow M_1[v] \to TRUE$ și $M_2[v] \to FALSE$
$\Rightarrow M_1$ acceptă $\forall v \in \Sigma^* \Rightarrow HALT(M, w) = TRUE$

Scanned with CamScanner

3. Istoria computațională pentru M[101] (adică configurațiile prin care trece) dată de tabelul stărilor:

	0	1	□	X
start	find1, X, →	find0, X, →	Y, □, -	start, X, →
find0	reset, X, ←	find0, 1, →	N, □, -	find0, X, →
find1	find1, 0, →	reset, X, ←	N, □, -	find1, X, →
reset	reset, 0, ←	reset, 1, ←	start, □, →	reset, X, ←

(ε, 101, start)
(X, 01, find0)
(ε, XX1, reset)
(ε, □XX1, reset)
(ε, XX1, start)
(X, X1, start)
(XX, 1, start)
(XXX, □, find0)
(XXX, □, N)

4. Complexitatea recurenței: $T(n) = 3T(\frac{n}{3}) + \log n$

Teorema Master:

$a = 3, b = 3$ $\quad f(n) = \log(n)$

$c = \log_b a = 1$ $\quad \log(n) \in O(n^{<1})$

↓

cazul I $\Rightarrow T(n) \in \Theta(n)$

5. Aduceți următoarea expresie la o formă mai simplă:

$$\frac{\Omega(n^2)}{O(n^3)} = \Omega\left(\frac{1}{n}\right)$$

$\exists c, n_0$ a.î. $\forall n \geq n_0$, $0 \leq cn^2 \leq f(n)$

$\exists c', n_0'$ a.î. $\forall n \geq n_0'$ $\quad 0 \leq g(n) \leq c'n^3 \quad | \cdot -1$

$\Rightarrow \frac{1}{c'n^3} \leq \frac{1}{g(n)} \quad | \cdot 0 \leq cn^2 \leq f(n) \Rightarrow \frac{cn^2}{c'n^3} \leq \frac{f(n)}{g(n)}$

$\Rightarrow 0 \leq c'' \cdot \frac{1}{n} \leq \frac{f(n)}{g(n)} \Rightarrow \Omega\left(\frac{1}{n}\right)$

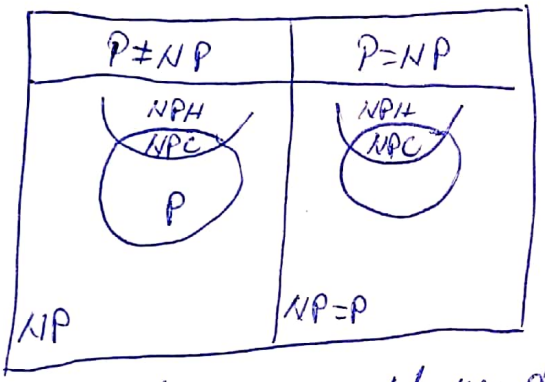
Scanned with CamScanner

6. Care este complexitatea următoarei secvențe?

```
sum = 0;
for (i = n/2; i <= n; i++){
    for (j = 1; j <= n; j = j * 2){
        sum = sum + n/2;
    }
}
```

for-ul extern se execută de $n/2$ ori (adunare)
for-ul intern se execută de $\log n$ ori (înmulțire) } ⇒ complexitate $\Theta(\frac{n}{2} \log n)$ sau $O(\frac{n}{2} \log n)$

7. f, g două probleme NP. Dacă $\exists$ o sol deterministic-polinomială pt f și $g \leq_p f$, ce putem spune despre cele două funcții?

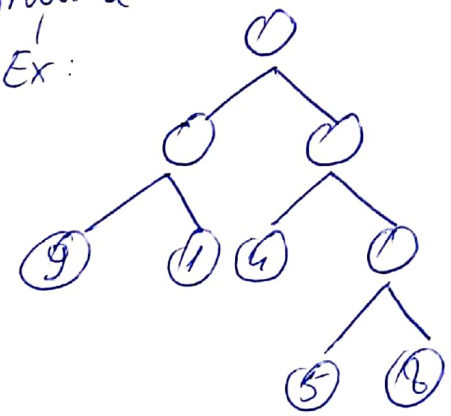


$f, g \in NP$
$f \in P$ } ⇒ $g \in P$
$g \leq_p f$

⇒ $\boxed{f, g \in P}$

8. Definiți un TDA pt un arbore binar în care doar frunzele pot conține valori.

Ex:



Leaf : N → Tree
Node : Tree × Tree → Tree
Node(Node(Leaf(9), Leaf(1)), Node(Leaf(4), Node(Leaf(5), Leaf(6))))

* or mai putea fi și Nil: —> Tree (în plus)

Scanned with CamScanner

9. ~~Demonstrați~~

Fie TDA-ul List studiat la curs, cu constructorii:

Empty: $\rightarrow$ List

Cons: $\mathbb{N} \times$ List $\rightarrow$ List

operatorii: length, append și reverse:

(L_1) length (Empty) = 0

(L_2) length (Cons (x, xs)) = 1 + length (xs)

(A_1) append (Empty, l) = l

(A_2) append (Cons (x, xs, l) = Cons (x, append (xs, l))

(R_1) reverse (Empty) = Empty

(R_2) reverse (Cons (x, xs)) = append (reverse (xs), Cons (x, Empty))

și teorema: (T_1) length (append (l_1, l_2)) = length (l_1) + length (l_2)

↓

toate astea ne sunt date

Dem. prin inducție structurală proprietatea:

$\forall l \in$ List, length (l) = length (reverse (l))

CB: l = Empty ✓

length (Empty) = length (reverse (Empty))

$\|(R_1)$

length (Empty)

Pi: length (l) = length (reverse (l))

length (cons (x, l)) = length (reverse (cons (x, l))

$\|(L_2)$ $\|(R_2)$

1 + length (l) = length (append (reverse (l), Cons (x, Empty)))

$\|(T_1)$

= length (reverse (l)) + length (Cons (x, Empty))

$\|(L_2)$

1 + length (Empty)

$\|(L_1)$

1

$\Rightarrow$ 1 + length (l) = length (reverse (l)) + 1

Scanned with CamScanner